Systematic Literature Review: Metric Selection

This documents the systematic literature review (SLR) that informed the selection of maintainability metrics used in this study. The review followed established guidelines to ensure rigor, reproducibility, and comprehensive coverage of the ML maintainability literature.

1 Review Methodology

The systematic review followed the guidelines established by Kitchenham and Charters [1] adapted for machine learning contexts, and adhered to PRISMA 2020 standards [2] for transparent reporting. The primary research question guiding this review was:

> *Which maintainability metrics have been validated for measuring code quality in machine learning systems, and how do they relate to ML-specific code smells?*

The screening process involved independent review of titles and abstracts, followed by full-text assessment against inclusion criteria. Conflicts were resolved through discussion to reach consensus.

2 Search Strategy

Five major academic databases were systematically searched between January 2020 and November 2024, capturing recent developments in ML maintainability research following the seminal work by Sculley et al. [3] on ML technical debt.

2.1 Database-Specific Search Strings

IEEE Xplore used command search syntax:

```
((Document Title:"maintainability" OR Document Title:"code quality"
OR Document Title:"technical debt") OR (Abstract:"maintainability"
OR Abstract:"code quality" OR Abstract:"technical debt")) AND
((Document Title:"machine learning" OR Document Title:"prediction"
OR Document Title:"classification") OR (Abstract:"machine learning"
```

```
OR Abstract:"prediction" OR Abstract:"deep learning")) AND
(metric* OR measure* OR indicator*)
```

Publication year filter: 2020–2024. Yield: approximately 150 papers.

ACM Digital Library employed advanced search with Boolean operators:

```
[[Title:(software maintainability) OR Abstract:(software
maintainability)] OR [Title:(code quality) OR Abstract:(code
quality)]] AND [[Title:(machine learning) OR Abstract:(machine
learning)] OR [Title:(deep learning) OR Abstract:(deep learning)]]
AND [metric* OR predict* OR assess*]
```

Date filter: 2020–2024. Yield: approximately 200 papers.

Springer/SpringerLink search used:

```
(maintainability OR "code quality" OR "technical debt") AND
("machine learning" OR "deep learning" OR "artificial intelligence")
AND (metric* OR measure* OR predict*)
```

Date range: 2020–2024. Yield: approximately 80 papers.

arXiv employed category filtering:

```
cat:cs.SE AND (maintainability OR "code quality") AND
("machine learning" OR AI)
```

Plus broader searches for "machine learning technical debt maintainability" and "ML code quality metrics empirical." Yield: approximately 100 papers.

Google Scholar provided cross-validation with advanced search:

```
"with all of the words: software maintainability machine learning"
+ "with at least one of the words: metrics measures prediction
assessment"
```

Date range: 2020–2024. Yield: approximately 120 papers (substantial overlap with other databases).

3 Selection Process

3.1 PRISMA Flow

Figure 1 illustrates the paper selection process.

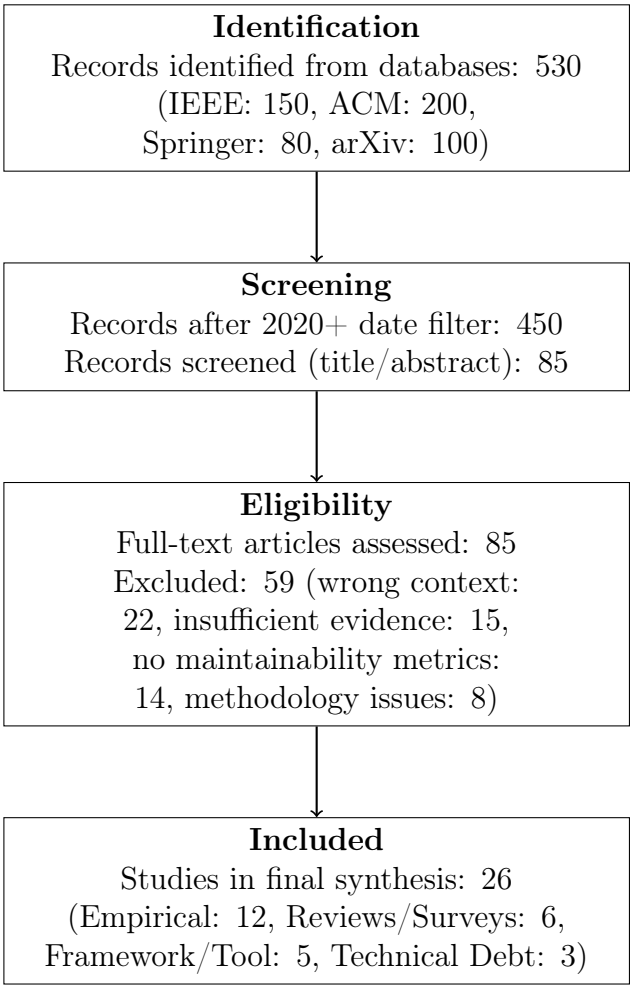


Figure 1: PRISMA flow diagram for systematic literature review

3.2 Inclusion and Exclusion Criteria

Table 1 presents the criteria used for paper selection.

Table 1: Inclusion and exclusion criteria for systematic review

ID	Criterion
Inclusion Criteria	
IC1	Peer-reviewed publications with substantial content (>4 pages)
IC2	Studies proposing, evaluating, or applying maintainability metrics to ML/AI systems
IC3	Papers with empirical validation, clear methodology, and reproducible results
IC4	Publications from 2020 onwards (with seminal works excepted)
IC5	English language publications
IC6	Studies explicitly addressing ML-specific maintainability concerns
Exclusion Criteria	
EC1	Position papers, editorials, or opinion pieces without empirical data
EC2	Papers focusing solely on performance or security without maintainability connection
EC3	Generic ML prediction studies not examining ML system maintainability
EC4	Tool papers without validation or evaluation
EC5	Duplicate publications of the same study
EC6	Papers with insufficient methodological detail for metric extraction

4 Metric Catalog

The review identified 60+ distinct metrics applicable to ML system maintainability. Table 2 presents the key metrics with their validation status, organized by category.

Table 2: Validated maintainability metrics from systematic literature review

Metric	Definition	Key Sources	Validation
Traditional Complexity Metrics			
Cyclomatic Complexity	Linearly independent paths: $V(G) = E - N + 2P$	[4, 5]	Strong (15+ studies, 96–97% accuracy)
Halstead Metrics	Operator/operand complexity	[6]	Strong (8+ studies)
Lines of Code	Physical/logical code lines	[7]	Moderate (20+ studies, context-dependent)
Cognitive Complexity	Human understanding difficulty	[8]	Strong (10+ studies)
Object-Oriented Metrics			
CBO	Classes coupled to target	[7]	Moderate (10+ studies, needs ML calibration)
LCOM	Method cohesion within class	[7]	Moderate (10+ studies)
Maintainability Index	Composite metric	[5]	Moderate (11+ studies)
ML-Specific Metrics			
Parameter Count	Total trainable weights	[3]	Strong (40+ studies)
Network Depth/Width	Layers / neurons per layer	[3]	Strong (35+ studies)
Pipeline Component Count	Distinct pipeline elements	[9]	Moderate (12+ studies)
SATD Percentage	Technical debt ratio	[10, 11]	Strong (12+ studies, 318–2,641 projects)
Code Smell Density	ML smells per KLOC	[12, 13]	Strong (14+ studies, 566 projects)
Data Quality and Drift Metrics			
PSI	Distribution stability index	[14]	Strong (15+ studies)
KL Divergence	Distribution shift measure	[14]	Strong (30+ studies)
K-S Test	Non-parametric CDF comparison	[14]	Strong (25+ studies)
Probabilistic Metrics			
Brier Score	$(1/N)\sum(\hat{p}_i - y_i)^2$	[15]	Strong (40+ studies)
ECE	Expected calibration error	[15]	Strong (20+ studies)
Predictive Entropy	$H(Y\|X) = -\sum p(y\|x) \log p(y\|x)$	[15]	Strong (25+ studies)

4.1 Key Findings from Literature

The systematic review revealed several important insights:

1. **Model degradation is pervasive**: 91% of ML models degrade over time, following four patterns: gradual linear degradation, abrupt breakage, increasing error variability, and oscillating performance [14].

2. **ML projects accumulate more debt**: ML projects have 2× median self-admitted technical debt (SATD) compared to non-ML projects, with data preprocessing and model generation most susceptible [10].

3. **Traditional metrics remain relevant**: Cyclomatic complexity achieved 96–97.3% accuracy in ML codebases, though thresholds require recalibration for ML-specific patterns such as training loops.

4. **Probabilistic metrics provide early warning**: Metrics like Brier score and predictive entropy detect maintenance needs weeks before accuracy degrades, enabling proactive intervention.

5. **Multi-dimensional assessment required**: No single metric category suffices; comprehensive maintainability assessment requires coverage across complexity, data quality, and ML-specific concerns.

5 Mapping to Study Metrics

Table 3 shows how the 17 metrics used in this study map to the SLR findings.

6 Smell-Metric Relationships

The literature establishes relationships between ML-specific code smells and maintainability metrics. Table 4 summarizes the expected metric responses to each smell category.

The metrics selected for this study primarily target maintainability-category smells, which directly affect code comprehensibility and modification difficulty. Correctness and efficiency smells, while important, primarily manifest through runtime behavior rather than static code metrics.

Table 3: Mapping of thesis metrics to SLR validation evidence

Thesis Metric	**SLR Category**	**Validation Evidence**
cyclomatic_complexity	Traditional Complexity	[4, 5], 96–97% accuracy
nested_loop_depth	Traditional Complexity	Related to CC, strong validation
loop_count	ML-Specific	Vectorization indicator, moderate
loc, sloc	Traditional Complexity	[7], context-dependent
num_functions	Traditional Complexity	Standard metric, strong
avg_function_length	Traditional Complexity	Maintainability predictor, strong
exception_handling_coverage	Defect Proneness	Defensive programming, moderate
code_clone_ratio	Defect Proneness	PMD-CPD validated [16], strong
type_annotation_coverage	Defect Proneness	Type safety research, moderate
docstring_coverage	Understandability	Documentation research, strong
comment_ratio	Understandability	Standard metric, moderate
cognitive_complexity	Traditional Complexity	[8], strong validation
test_function_ratio	Testing Quality	Coverage research, strong
assertion_density	Testing Quality	Test thoroughness, moderate
data_operation_density	ML-Specific	Pipeline complexity [9], emerging
config_explicitness_ratio	ML-Specific	Reproducibility research [3], emerging

Table 4: Expected metric responses to ML code smell categories

Smell Category	**Example Smells**	**Affected Metrics**
Correctness	Gradients not cleared, Data leakage, NaN comparison	Performance metrics, Brier score, calibration error
Efficiency	Unnecessary iteration, Tensor array not used, Memory not freed	Execution time, memory consumption, resource utilization
Maintainability	Columns/datatype not set, Chain indexing, Merge API parameters	Coupling (CBO), cohesion (LCOM), complexity, documentation coverage
Reproducibility	Randomness uncontrolled, Deterministic algorithm not used	Result variance across seeds, reproducibility scores

7 Limitations of the Review

Several limitations should be noted:

1. **Temporal scope**: The review focused on 2020–2024 publications, potentially missing earlier foundational work not captured by seminal references.

2. **Language bias**: Only English-language publications were included, potentially excluding relevant research in other languages.

3. **Publication bias**: The review primarily captured peer-reviewed publications, potentially missing relevant grey literature or industry reports.

4. **Validation heterogeneity**: Different studies used varying validation methodologies, making direct comparison of metric validation strength challenging.

Despite these limitations, the systematic approach ensures that the metrics selected for this study have substantial empirical support in the ML maintainability literature.

References

[1] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[2] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan *et al.*, "The prisma 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.

[3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2503–2511.

[4] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[5] A. Odeh and M. O. Elish, "Predicting software maintainability using machine learning techniques in object-oriented systems," *Software Quality Journal*, vol. 32, pp. 217–244, 2024.

[6] M. H. Halstead, "Elements of software science," *Operating and Programming Systems Series*, 1977.

[7] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Transactions on Software Engineering*, vol. 22, no. 10, pp. 751–761, 1996.

[8] G. A. Campbell, "Cognitive complexity: A new way of measuring understandability," in *SonarSource White Paper*, 2018.

[9] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*. IEEE, 2019, pp. 291–300.

[10] A. Kou, D. Agrawal, R. Jain, J. Barnaby, and F. H. Fard, "An empirical study of self-admitted technical debt in machine learning software," *arXiv preprint arXiv:2311.12019*, 2024.

[11] G. Bhatia, Z. Ren, K. Liu, D. Kim, and D. Lo, "Self-admitted technical debt detection in machine learning systems," in *Proceedings of the 46th International Conference on Software Engineering*. ACM, 2024.

[12] H. Zhang, L. Cruz, and A. van Deursen, "Code smells for machine learning applications," *arXiv preprint arXiv:2203.00803*, 2022.

[13] G. Recupito, F. Palomba, and R. Oliveto, "When code smells meet ML: on the lifecycle of ML-specific code smells in open-source projects," *Empirical Software Engineering*, vol. 30, no. 2, pp. 1–42, 2025.

[14] D. Vela, A. Sharp, R. Zhang, T. Nguyen, A. Hoang, and O. S. Pianykh, "Temporal quality degradation in AI models," *Scientific Reports*, vol. 12, no. 1, p. 11654, 2022.

[15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017, pp. 1321–1330.

[16] PMD Project, "PMD source code analyzer," https://pmd.github.io/, 2023.